

8085 Microprocessor Assembly Language Programming: Selected Algorithms and Implementations

Microprocessor Systems Coursework

May 10, 2026

Abstract

This document outlines standard 8085 assembly language implementations for various data processing, arithmetic, and hardware interface operations. Each section details the algorithm and provides the corresponding assembly code.

1 Array Deduplication

Problem: Write an assembly language program to delete repeated numbers from a given array of N elements and store the resultant numbers in a new array.

Logic: We iterate through the source array. For each element, we scan the newly built destination array. If the element is not found, we append it to the destination array.

Listing 1: Remove Repeated Numbers from Array

```
1 ; Source array starts at 2001H, length N is at 2000H
2 ; Destination array starts at 3001H, length is stored at 3000H
3     LXI H, 2000H      ; HL points to source length
4     MOV C, M          ; C = N (outer loop counter)
5     INX H             ; HL points to first element
6     LXI D, 3000H      ; DE points to destination length
7     MVI B, 00H        ; B = length of new array (initially 0)
8
9 NEXT:  MOV A, M        ; Load current array element into A
10      PUSH H           ; Save source pointer
11      PUSH B           ; Save loop counters
12
13      ; Check if A exists in destination array
14      LXI H, 3001H     ; HL points to start of new array
15      MOV B, B          ; Check if new array is empty
16      JZ ADD_IT        ; If empty, directly add
17
18 CHECK: CMP M          ; Compare current element with new array
      element
```

```

19      JZ SKIP          ; If match found, skip adding
20      INX H           ; Next element in new array
21      DCR B           ; Decrement inner counter
22      JNZ CHECK       ; Loop until end of new array
23
24 ADD_IT: POP B         ; Restore loop counters
25      POP H           ; Restore source pointer
26      PUSH H          ; Save source pointer again
27      LXI H, 3001H    ; HL points to new array data
28      MOV E, B        ; Calculate offset
29      MVI D, 00H
30      DAD D           ; HL = 3001H + current length
31      MOV M, A        ; Store element in new array
32      INR B           ; Increment new array length
33      JMP CONTINUE
34
35 SKIP:  POP B         ; Restore registers if skipped
36      POP H
37
38 CONTINUE:
39      INX H           ; Point to next element in source
40      DCR C           ; Decrement outer loop counter
41      JNZ NEXT        ; Loop until all source elements are checked
42
43      ; Store final length of new array
44      MOV A, B
45      STA 3000H       ; Store length of destination array
46      HLT            ; Terminate Program

```

2 14-bit Hexadecimal to 4-digit BCD

Problem: Write an assembly language program to convert a 14-bit Hexadecimal data into a 4-digit Binary Coded Decimal (BCD) value.

Logic: A 14-bit Hex value has a maximum decimal equivalent of 16383. To fit within a 4-digit BCD (9999), we assume the input $\leq 270FH$. We use repeated subtraction of 1000 (03E8H), 100 (0064H), and 10 (000AH).

Listing 2: Hexadecimal to BCD Conversion

```

1 ; Input Hex at 2000H (LSB) and 2001H (MSB)
2 ; Output BCD at 3000H (LSB, Tens/Ones) and 3001H (MSB, Thousands/
  Hundreds)
3      LHLD 2000H      ; Load 16-bit Hex into HL
4      LXI B, 03E8H    ; BC = 1000
5      CALL DIVIDE     ; A = quotient (Thousands)
6      MOV D, A        ; D = Thousands digit
7

```

```

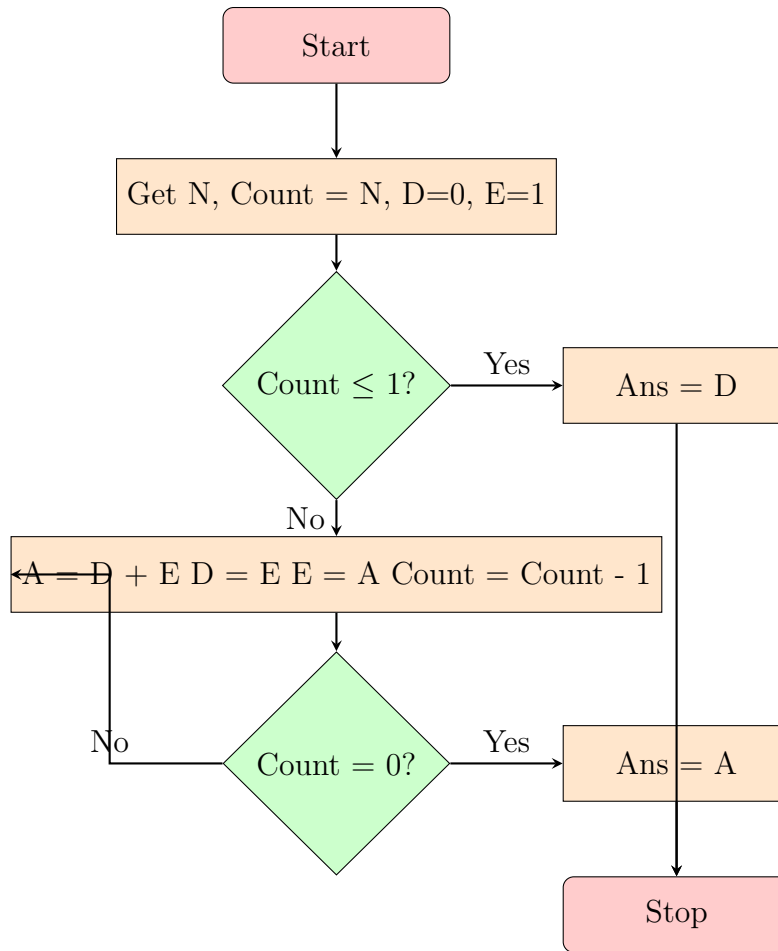
8      LXI B, 0064H      ; BC = 100
9      CALL DIVIDE      ; A = quotient (Hundreds)
10     RLC
11     RLC
12     RLC
13     RLC              ; Shift to upper nibble
14     ORA D            ; Combine Thousands and Hundreds
15     STA 3001H        ; Store MSB of BCD
16
17     LXI B, 000AH      ; BC = 10
18     CALL DIVIDE      ; A = quotient (Tens)
19     RLC
20     RLC
21     RLC
22     RLC              ; Shift to upper nibble
23     MOV D, A          ; Save Tens in upper nibble
24     MOV A, L          ; Remainder in L is Ones
25     ORA D            ; Combine Tens and Ones
26     STA 3000H        ; Store LSB of BCD
27     HLT
28
29     ; Subroutine for 16-bit division by repeated subtraction
30 DIVIDE: MVI A, 00H    ; Initialize quotient
31 DIV_LP: MOV E, A      ; Save quotient temporarily
32         MOV A, L      ; HL - BC
33         SUB C
34         MOV L, A
35         MOV A, H
36         SBB B
37         MOV H, A
38         JC DIV_END    ; If borrow, HL < BC
39         MOV A, E
40         INR A          ; Increment quotient
41         JMP DIV_LP
42 DIV_END:
43         DAD B          ; Add back BC to restore remainder
44         MOV A, E
45         RET            ; A = final quotient

```

3 Fibonacci Sequence Generation

Problem: Write an 8085 Assembly language program to find the Fibonacci number of an 8-bit number. Draw the flow chart of the algorithm used.

Flowchart:



Listing 3: Nth Fibonacci Number

```

1 ; Input N at 2000H, Output at 2001H
2   LDA 2000H           ; Load N
3   MOV C, A            ; C = counter
4   MVI D, 00H          ; D = F(0)
5   MVI E, 01H          ; E = F(1)
6
7   CPI 01H
8   JC ZERO_CASE        ; If N=0
9   JZ ONE_CASE         ; If N=1
10
11  DCR C                ; Counter runs N-1 times
12 LOOP: MOV A, D        ; A = F(n-2)
13      ADD E            ; A = F(n-2) + F(n-1)
14      MOV D, E         ; F(n-2) becomes old F(n-1)
15      MOV E, A         ; F(n-1) becomes new sum
16      DCR C
17      JNZ LOOP
18
19  STA 2001H           ; Store result

```

```

20         HLT
21
22 ZERO_CASE:
23     MOV A, D
24     STA 2001H
25     HLT
26
27 ONE_CASE:
28     MOV A, E
29     STA 2001H
30     HLT

```

4 Factorial Using Subroutine

Problem: Compute the factorial of a number (between 1 to 6) using the subroutine method.

Listing 4: Factorial Using Multiplication Subroutine

```

1  ; Input at 2000H, Output at 2001H
2      LXI SP, FFFFH      ; Initialize Stack Pointer
3      LDA 2000H          ; Load N
4      MOV B, A           ; B = N
5      MVI C, 01H        ; C will hold the factorial result, init to
6                          1
7
8 FACT_LP:
9      CALL MULTIPLY      ; C = C * B
10     DCR B              ; N = N - 1
11     JNZ FACT_LP        ; Loop until B = 0
12
13     MOV A, C
14     STA 2001H          ; Store Result
15     HLT
16
17 ; Subroutine to multiply C and B, result in C
18 MULTIPLY:
19     MOV D, B           ; D = counter for addition
20     MOV E, C           ; E = multiplicand
21     MVI A, 00H         ; Accumulator = 0
22     MUL_LP: ADD E       ; A = A + multiplicand
23     DCR D
24     JNZ MUL_LP
25     MOV C, A           ; Save result back to C
26     RET

```

5 Pulse Generation via Interrupt

Problem: Generate a pulse of duration 1 ms via I/O port 01H initiated by pressing a push button connected to the RST 7.5 vector interrupt pin.

Logic: Enable the RST 7.5 hardware interrupt. In the Interrupt Service Routine (ISR) mapped at 003CH, generate a logic high, execute a 1ms delay loop, and then return to logic low.

Listing 5: 1ms Pulse on RST 7.5

```
1 ; Main Program
2     ORG 0000H
3     LXI SP, 27FFH    ; Initialize SP
4     EI               ; Enable Interrupts
5     MVI A, 0BH       ; Unmask RST 7.5
6     SIM              ; Set Interrupt Mask
7 HERE:    JMP HERE    ; Wait for interrupt indefinitely
8
9 ; Interrupt Service Routine for RST 7.5
10    ORG 003CH        ; Vector address for RST 7.5
11    MVI A, 01H        ; Pulse HIGH
12    OUT 01H           ; Send to Port 01H
13    CALL DELAY_1MS    ; Wait 1ms
14    MVI A, 00H        ; Pulse LOW
15    OUT 01H           ; Send to Port 01H
16    EI               ; Re-enable interrupts
17    RET              ; Return to main program
18
19 ; Delay Subroutine (Assumes 2 MHz Clock)
20 ; T-states required for 1ms = 2000
21 DELAY_1MS:
22     MVI C, 8BH       ; Hex 8B = 139 decimal
23 D_LOOP: DCR C        ; 4 T-states
24     JNZ D_LOOP       ; 10/7 T-states
25     RET              ; 10 T-states
```

6 Highest Common Factor (HCF)

Problem: Find the HCF of two 8-bit numbers stored at 8000H and 8001H. Display the result in 8050H.

Logic: We utilize the Euclidean algorithm by repeated subtraction. We continuously subtract the smaller number from the larger number until both values are equal. The equal value is the HCF.

Listing 6: HCF by Repeated Subtraction

```
1 ; Numbers at 8000H and 8001H, Result at 8050H
2     LXI H, 8000H    ; Point to first number
```

```

3      MOV A, M          ; A = Num 1
4      INX H
5      MOV B, M          ; B = Num 2
6
7  COMPARE:
8      CMP B             ; Compare A and B
9      JZ FOUND_HCF      ; If A = B, HCF is found
10     JC A_IS_LESS      ; If A < B, jump to A_IS_LESS
11
12     ; A > B, so A = A - B
13     SUB B
14     JMP COMPARE
15
16  A_IS_LESS:
17     ; B > A, so B = B - A
18     MOV C, A           ; Save A in C temporarily
19     MOV A, B           ; Move B to A for subtraction
20     SUB C              ; A = B - A
21     MOV B, A           ; Store result back in B
22     MOV A, C           ; Restore A
23     JMP COMPARE
24
25  FOUND_HCF:
26     STA 8050H          ; Store HCF result
27     HLT               ; Terminate

```